LLM-CIM: A 28nm 126.7TOPS/W Input-LUT-based Digital CIM Macro with Reconfigurable Matrix Multiplication and Nonlinear Operation Modes for LLMs

Yiqi Wang[1], Zhen He[1], Zihan Wu[1], Ruiqi Guo[1], Longke Yan[2], Huiming Han[1], Yang Wang[1], Shaojun Wei[1], Yang Hu[1], Fengbin Tu[2*], Shouyi Yin[1*]

[1]Tsinghua University, Beijing; [2]The Hong Kong University of Science and Technology, Hong Kong; *Corresponding Author

Abstract

This paper presents a Digital Computing-in-Memory (DCIM) macro tailored for Large Language Model (LLM) acceleration, named LLM-CIM. It has three key features: 1) A Maximum Reuse-Oriented Input-LUT (MRIL) CIM reduces compute logic area and power by 40.7% and 37.9%. 2) An Outlier Aggregation-based Reordering Unit (OARU) saves 1.68~1.72x CIM computation time. 3) A Normalized-Domain Computation Converter (NDCC) improves resource utilization to 95.6~98.7%. LLM-CIM achieves 126.7TOPS/W energy efficiency on LLaMA2-7B, with up to 5.36x energy saving over SOTA CIMs.

Introduction

LLMs are gaining widespread attention [1], while their large model sizes incur massive data movement and computation. DCIM is a promising architecture for reducing data movement [2]. Recent advances in look-up table (LUT)-based DCIM [3,4] aim to reduce DCIM's adder tree overhead by pre-calculating weight sums once using First-level Adders (FLAs) in each weight-LUT, so each input bit can reuse the LUT for multiply-accumulation (MAC), amortizing the FLA overhead. However, LLM's computation requirements for high compute efficiency and hybrid input bitwidths in Matrix Multiplications (MMs) and varied Non-linear Operation (NL) ratios raise new challenges for DCIM (Fig.1&2): 1) The weight-LUT count is proportional to the MAC unit count in a CIM row, incurring an increased number of FLAs as each LUT requires an FLA. 2) Input matrices in LLMs have scattered outlier channels with much larger magnitudes than other channels, difficult to quantize into low bitwidth [5]. Quantizing outliers and normal values separately into INT8 and INT4 [6] lead to imbalanced bitwidths, wasting CIM computation time. 3) Conventional separate CIM and NL unit suffer from fixed MM and NL performance ratio, failing to adapt to varied workload ratios under diverse token lengths [7]. Unequal pipeline stage durations cause resource underutilization.

LLM-CIM Overall Architecture

LLM-CIM comprehensively tackles the above challenges with 3 features in two modes: 1) A MRIL CIM exploits the higher reuse time of inputs than weights due to input broadcasting in CIM, and pre-calculates only one input sum via a FLA in single input-LUT, shared by all MAC units. 2) An OARU defers outlier processing to the end via dynamic input reorder, so aggregated outliers can be computed together after calculating all normal values, ensuring CIM simultaneously processing inputs with the same bitwidths to save computation time. 3) An NDCC decomposes NL computation with a wide input range into simple mathematical transformations and linear interpolation for normalized-domain inputs with a reduced range that can be easily realized on CIM. Thus, in spatial mapping of multiple CIMs, each dual-mode CIM can adaptively reconfigure as either MM or NL mode, to match varied workload ratios and improve utilization. Fig.3 shows its overall architecture. Each CIM core reconfigures modes by selectively activating 4 OARUs or NDCCs as the inputs of 4 MRIL CIM macros. In MM mode, the quantized outliers are held in outlier FIFOs and sent to CIM after all normal FIFOs' elements are consumed. The input pre-calculator generates the input sum and broadcasts it across all weights in a CIM row. In NL mode, the normalized inputs are sent to 4 subarrays for parallel linear interpolation, whose results are finally transformed back to the original input range.

Fig.4&5 detail the **Maximum Reuse-Oriented Input-LUT** CIM macro, reducing compute logic overhead. A MAC operation between two sets of weights and inputs can be split based on the binary encoding of inputs or weights. TSMC's DCIM [3,4] splits each MAC result by input bit, and uses LUTs to implement each partial sum i^{th} Sum_j. The two adjacent weights' sum $W_{0,j}+W_{1,j}$ is pre-calculated in each LUT, and each partial sum result is selected from LUT based on $I_0[i]$ $I_1[i]$. However, each weight output channel (Och) requires a separate LUT, and each LUT's reuse time is limited to the input bitwidth (e.g. 4). In contrast, we split MAC results by weight bit, and pre-calculate I_0+I_1 in one input-LUT. As all weights in a row are multiplied with the same input, the single LUT can be shared across all weight Ochs' partial sums, largely raising the LUT's reuse time compared to weight-LUT method (from 4 to 24 in Fig.3), and reducing the FLA number from Och count to one. Two adjacent weights' bits (e.g. $W_{0,0}[3]$ $W_{1,0}[3]$) are coalesced in a CIM row as multiplexers' select signals for ease of implementation. 4 partial sums are shift-added to derive the final INT4 MAC result. Compared to traditional [2] and weight-LUT [3,4] DCIM, MRIL CIM saves 40.7% and 13.3% area, 37.9% and 12.4% power respectively.

Fig.6&7 depict the **Outlier Aggregation-based Reordering Unit**, saving CIM computation time. Each subarray activates Row0 to Row7 in turn, but an outlier input in any subarray makes CIM take 2 cycles for computation. To synchronize subarrays' outlier processing, we aggregate outlier and normal channels in each subarray's input submatrix, so all subarrays' inputs are of one type at a time, either all normal values or outliers. Since outlier channels can be identified using calibration data [8], the desired channel order is known offline, guiding collaborative weight and input reorder to ensure computation equivalence. Weights are statically reordered in CIM initialization. Outlier channel corresponding weights are all placed on the last row of each subarray. Inputs need to be reordered online by OARU, as they are dynamically generated on chip from previous computation. After separate quantization for the outlier and normal values based on the known bitmap, two quantized values are dispatched to 2 FIFOs to aggregate normal values and outliers for each subarray. Outlier FIFOs are popped until the subarrays' last rows are activated, enabling OARU to output sequential inputs in the desired channel order. OARU achieves 1.68~1.72x speedup.

Fig.8&9 show the **Normalized-Domain Computation Converter**, enabling CIM with NL capabilities to improve resource utilization. Linear interpolation is widely used to approximate NLs [9], dividing the input range into segments and computing Kx+B for each segment. Typical NLs in LLM can be expressed as a combination of 1/Sqrt, Div, and Exp, but their wide input range necessitates multiple segments for accurate approximation, leading to numerous interpolation parameters that are difficult to store in CIM. To address the issue, NDCC extracts the input's mantissa [1,2) or decimal [0,1) part and sends it to NL mode CIM, allowing it to store only a few parameters for inputs in the normalized domain instead of all possible ranges. We reformulate Kt+B as the sum of two INT4 MACs, so compute logic for MM can be reused for linear interpolation in NL mode. The output denormalizer finally recovers the original input's results. Since mathematical identity transformations ensure the normalized-domain interpolations accurately approximate NL results from the original input range, our method largely reduces the required segment number without accuracy loss. NDCC saves 74.2% area, and the dual-mode CIM achieves up to 98.7% utilization.

Measurement Results

The fabricated LLM-CIM can work at 0.6-1.0V supply, 85-410 MHz (Fig.10). The peak MM efficiency for hybrid INT4/8 inputs and INT4 weights on LLaMA2-7B is 126.7TOPS/W. LLM-CIM consumes only 0.51J for LLaMA2-7B, reducing energy by 3.96x over prior LUT-based CIM [10] and 5.36x over transformer-targeted CIM [11] (Fig.11). Although the more advanced 3nm DCIM [4] has higher INT efficiency, it can only process hybrid INT4/8 inputs using INT8 precision, while LLM-CIM achieves 1.42x higher energy efficiency thanks to the hybrid precision support and improved resource utilization.

Acknowledgements: This work was supported in part by National Key Research and Development Program (2023YFB4403100), NSFC Grant 62422407, 62125403, 92164301 and 62304121, the Science and Technology Innovation 2030 Grant 2021ZD0114402, the BNRist, and the Beijing Advanced Innovation Center for Integrated Circuits

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:44:26 UTC from IEEE Xplore. Restrictions apply.

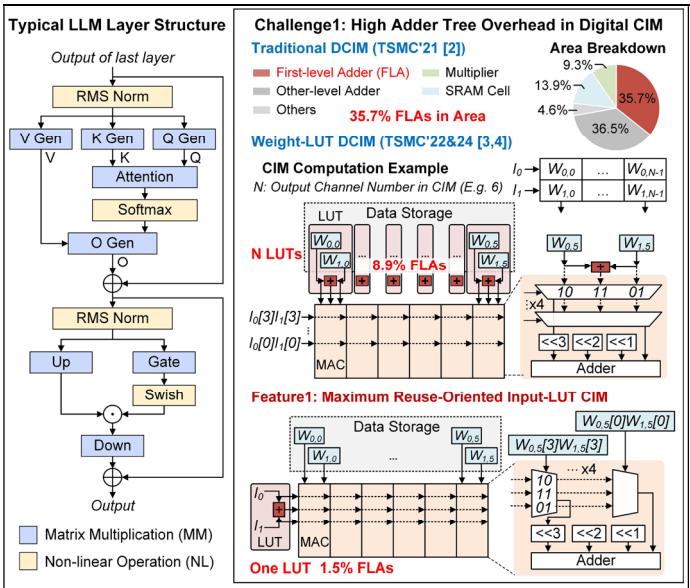


Fig. 1. Typical LLM layer structure, the 1st challenge of designing a digital CIM for LLMs, and LLM-CIM's Feature1 (MM Mode).

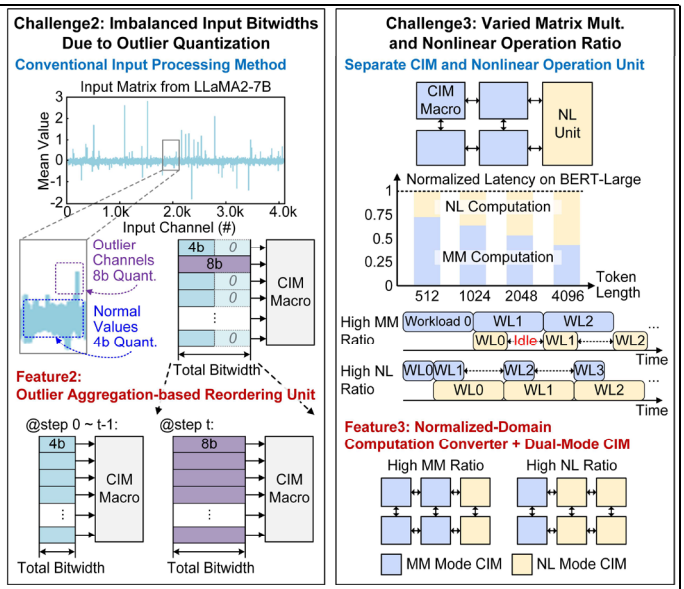


Fig. 2. The 2nd and 3rd challenges of designing a DCIM for LLMs, and LLM-CIM's Feature2 (MM Mode) and Feature3 (NL Mode).

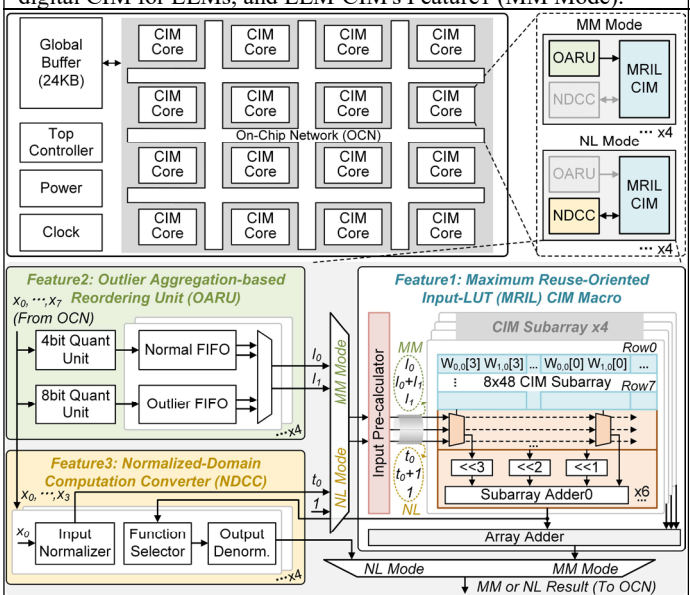


Fig. 3. LLM-CIM's overall architecture and configuration for Matrix Multiplication (MM)/ Nonlinear Operation (NL) modes.

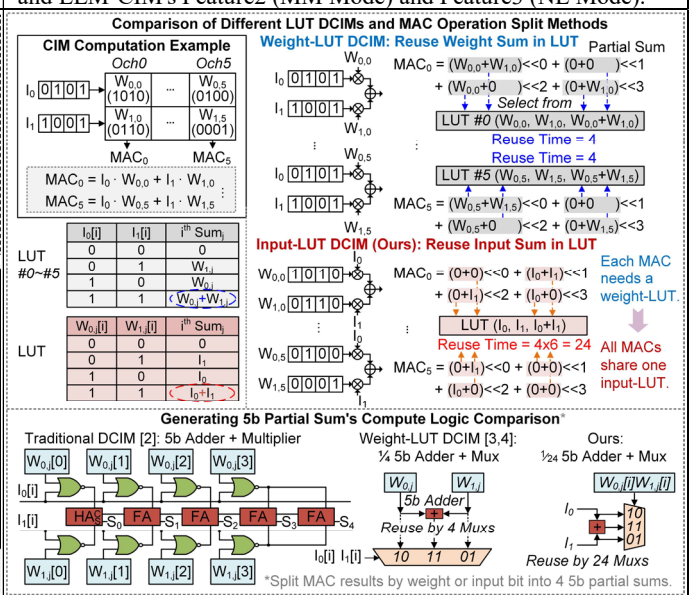


Fig. 4. Maximum Reuse-Oriented Input-LUT (MRIL) CIM method, pre-calculating input sum and sharing it across all weights in a row.

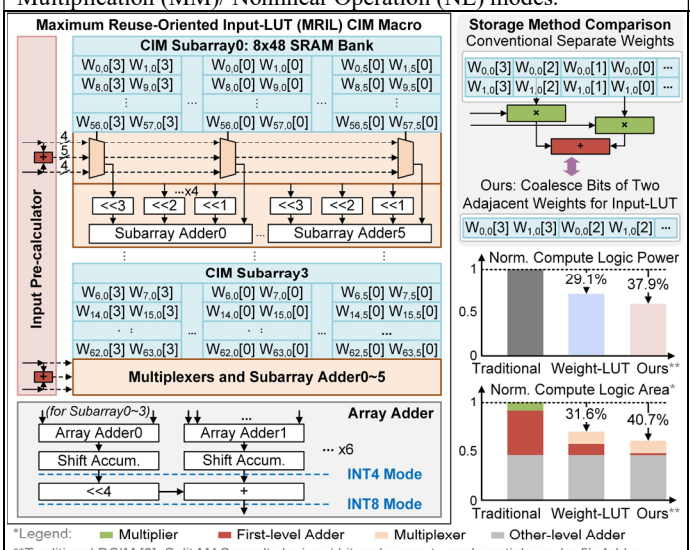


Fig. 5. MRIL CIM architecture and experiments, reducing compute logic power and area overhead.

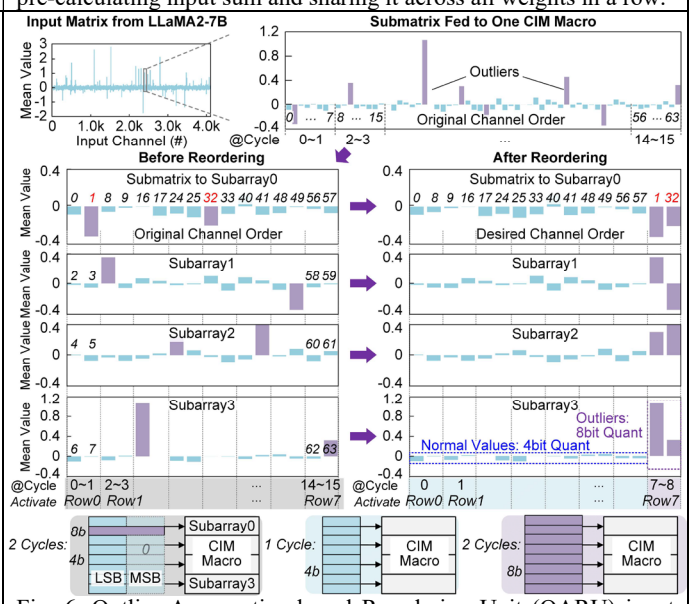


Fig. 6. Outlier Aggregation-based Reordering Unit (OARU) input reorder example, aggregating normal value and outlier processing.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:44:26 UTC from IEEE Xplore. Restrictions apply.

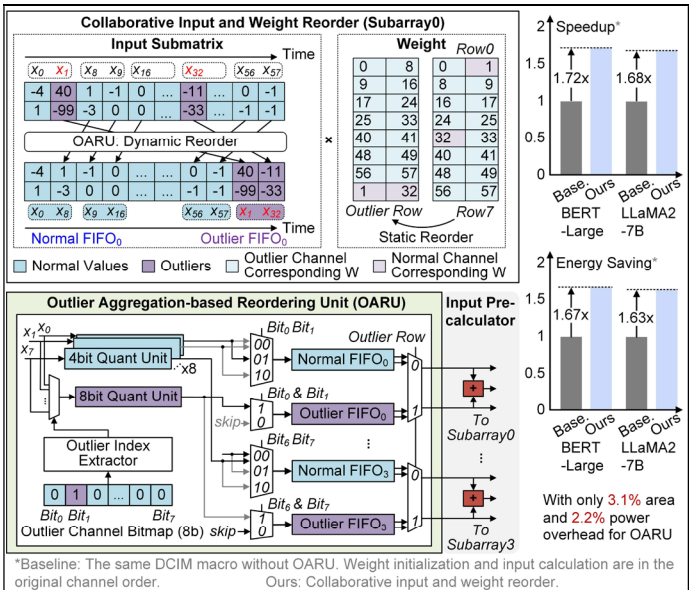


Fig. 7. OARU method, architecture, and experiments, supporting hybrid input bitwidths and saving CIM computation time.

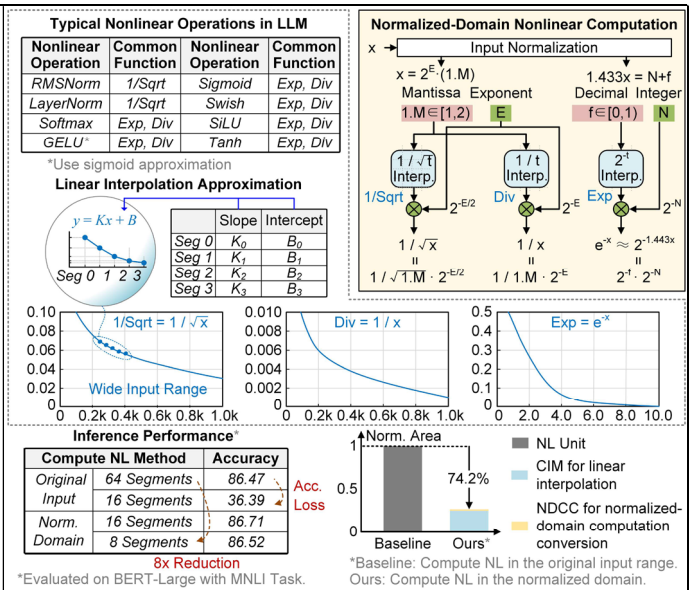


Fig. 8. Normalized-Domain Computation Converter (NDCC) method that converts NL computation to normalized-domain interpolations.

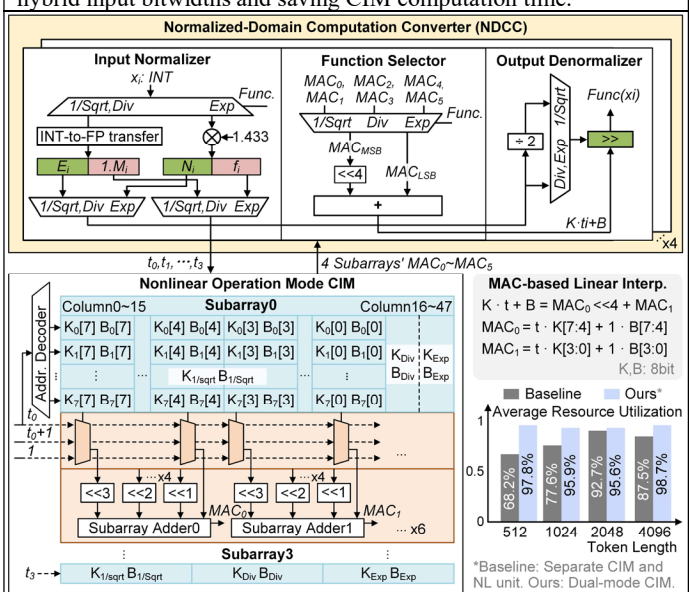
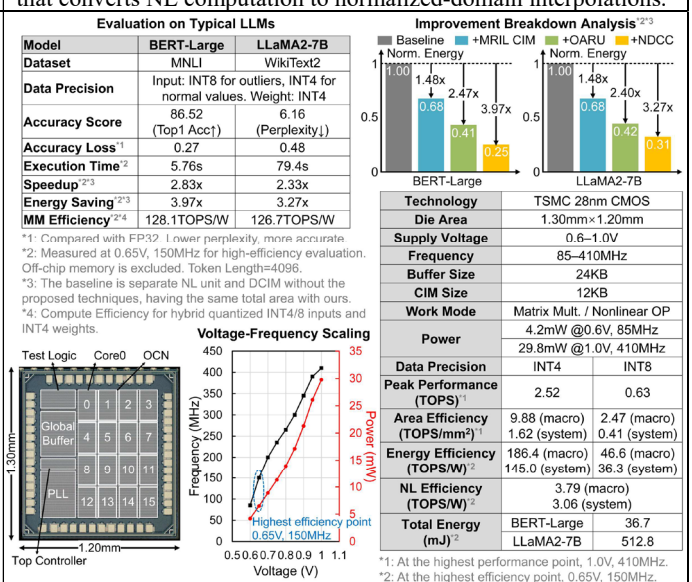


Fig. 9. NDCC architecture and experiments, enabling CIM with NL capabilities to improve utilization via adaptive mode reconfiguration.

Evaluation on Typical LLMs

Model	BERT-Large	LLaMA2-7B
Dataset	MNLI	WikiText2
Data Precision	Input: INT8 for outliers, INT4 for normal values. Weight: INT4	
Accuracy Score	86.52 (Top1 Acc↑)	6.16 (Perplexity↓)
Accuracy Loss*1	0.27	0.48
Execution Time*2	5.76s	79.4s
Speedup*2*3	2.83x	2.33x
Energy Saving*2*3	3.97x	3.27x
MM Efficiency*2*4	128.1TOPS/W	126.7TOPS/W

*1: Compared with FP32. Lower perplexity, more accurate.
*2: Measured at 0.65V, 150MHz for high-efficiency evaluation. Off-chip memory is excluded. Token Length=4096.
*3: The baseline is separate NL unit and DCIM without the proposed techniques, having the same total area with ours.
*4: Compute Efficiency for hybrid quantized INT4/8 inputs and INT4 weights.



Technology	TSMC 28nm CMOS
Die Area	1.30mm×1.20mm
Supply Voltage	0.6–1.0V
Frequency	85–410MHz
Buffer Size	24KB
CIM Size	12KB
Work Mode	Matrix Mult. / Nonlinear OP
Power	4.2mW @0.6V, 85MHz 29.8mW @1.0V, 410MHz

Data Precision	INT4	INT8
Peak Performance (TOPS)*1	2.52	0.63
Area Efficiency (TOPS/mm²)*1	9.88 (macro) 1.62 (system)	2.47 (macro) 0.41 (system)
Energy Efficiency (TOPS/W)*2	186.4 (macro) 145.0 (system)	46.6 (macro) 36.3 (system)
NL Efficiency (TOPS/W)*2	3.79 (macro) 3.06 (system)	
Total Energy (mJ)*2	BERT-Large	36.7
	LLaMA2-7B	512.8

*1: At the highest performance point, 1.0V, 410MHz.
*2: At the highest efficiency point, 0.65V, 150MHz.

Fig. 10. Measurement results, die photo, voltage-frequency scaling curves, and summary table.

TABLE I. Comparison with state-of-the-art CIM macros

	VLSI'22 C2.5 [3]	ISSCC'24-34.4 [4]	ISSCC'24-34.7 [10]	ISSCC'24-34.5 [11]	This Work
CIM Type	SRAM-based Digital CIM	SRAM-based Digital CIM	eDRAM-based Digital CIM	SRAM-based Charge-Domain CIM	**SRAM-based Digital CIM**
Data Precision	INT4/8	INT12	INT8	INT4-8	**√ Hybrid INT4/8**
FLA Optimization	Reuse Weight Sum	Reuse Weight Sum	Reuse Weight Sum*1	×	**√ Reuse Input Sum**
Work Mode	MM	MM	MM	MM	**√ MM / NL**
Technology	12nm	3nm	28nm	65nm	**28nm**
Supply Voltage (V)	0.72	0.36-1.1	0.9-1.2	0.6-1.1	**0.6-1.0**
Frequency (MHz)	N/A	250-1600	400-1100	N/A	**85-410**
Die Area (mm²)	0.032 (macro)	0.016 (macro)	3.24	0.48	**1.56**
Power (mW)	11.2	26.6	64.6	1.47	**4.2-29.8**
Peak Performance (TOPS) INT4	1.34	7.78	0.07	0.08	**2.52***2
Peak Performance (TOPS) INT8	0.34	1.94	0.28	0.02	**0.63***2
Energy Efficiency (TOPS/W) INT4	121.0	293.0	72.4	51.1	**145.0***3
Energy Efficiency (TOPS/W) INT8	30.3	73.3	18.1	12.8	**36.3***3
NL Efficiency (TOPS/W)*4	N/A	N/A	N/A	N/A	**3.06***3
Energy Consumption*5 on LLaMA2-7B (J)	MM: 60.5 TOPS/W Total: 1.33 (2.60x)	MM: 146.5 TOPS/W Total: 0.73 (1.42x)	MM: 36.2 TOPS/W Total: 2.03 (3.96x)	MM: 25.6 TOPS/W Total: 2.75 (5.36x)	**MM: 126.7 TOPS/W Total: 0.51 (1x)***3

*1: Use 4x eDRAM cells to store 16 combinations of 4 weights.
*2: Measured at the highest performance point, 1.0V, 410MHz.
*3: Measured at the highest efficiency point, 0.65V, 150MHz.
*4: One OP = one nonlinear operation.
*5: Enhance other architectures with a NL unit to support all LLM computations.

References

[1] T. Brown *et al.*, *NeurIPS*, 2020, pp. 1877-1901.
[2] Y. Chih *et al.*, *ISSCC*, 2021, pp. 252–253.
[3] C.-F. Lee *et al.*, *Symp. on VLSI*, 2022, pp. 24–25.
[4] H. Fujiwara *et al.*, *ISSCC*, 2024, pp. 572-574.
[5] G. Xiao *et al.*, *ICML*, 2023, pp. 38087-38099.
[6] W. Li *et al.*, *TCAS-I*, 2024, pp. 2858-2871.
[7] J. R. Stevens *et al.*, *DAC*, 2021, pp. 469-474.
[8] T. Dettmers *et al.*, *NeurIPS*, 2022, pp. 30318-30332.
[9] S. Lee *et al.*, *ISSCC*, 2022, pp. 1-3.
[10] Y. He *et al.*, *ISSCC*, 2024, pp. 578-580.
[11] K. Yoshioka *et al.*, *ISSCC*, 2024, pp. 574-576.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:44:26 UTC from IEEE Xplore. Restrictions apply.